



UAC

Universidad Autónoma
de Campeche

"Del enigma sin albas a triángulos de luz"

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

FACULTAD DE INGENIERÍA

LIC. EN INGENIERÍA EN TECNOLOGÍA DE SOFTWARE

Unidad de aprendizaje: (5to. Semestre Grupo A ITS)

Alumno: Joaquín Alberto de la Cruz Morales Fuentes

Nombre del trabajo: Practica 01 - Sección Crítica

Fecha de entrega: 28 de agosto de 2026

Profesor: Sergio A Noh Puch

Condiciones de carrera y sección crítica

En esta práctica se estudió el comportamiento de múltiples hilos cuando acceden y modifican una variable compartida. El objetivo principal fue comprender el concepto de **condición de carrera**, identificar una **sección crítica** y observar cómo un mecanismo de sincronización, en este caso un **mutex**, permite evitar problemas causados por el acceso concurrente.

La variable utilizada fue:

```
int global_counter = 20;
```

El primer hilo realiza:

```
global_counter++;
```

mientras que el segundo realiza:

```
global_counter--;
```

Decidí ejecutar la operación millones de veces para aumentar las posibilidades de observar los efectos de la ejecución concurrente.

Implementación sin mutex

Primero se implementó el programa sin utilizar ningún mecanismo de sincronización:

El resultado esperado matemáticamente sería:

$$20 + 10000000 - 10000000 = 20$$

Por lo tanto, si las operaciones fueran completamente independientes y no existieran problemas de concurrencia, el resultado siempre debería ser 20.

Sin embargo, al ejecutar el programa sin mutex se observaron resultados variables.

```
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -131262
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -29826
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -572122
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -1033096
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -455064
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -2223095
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -2274429
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -527946
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = 1453999
joaquin@LAPTOP-8G6CD4RU:/t
lo 1$ ./race.exe
global_counter = -3406854
```

Implementación utilizando mutex

Posteriormente se agregó un mutex para controlar el acceso a la variable compartida.

La diferencia principal es que ahora cada modificación de `global_counter` está protegida por:

```
pthread_mutex_lock(&mutex);
```

y:

```
pthread_mutex_unlock(&mutex);
```

El lock solicita el acceso exclusivo al mutex. Si otro hilo ya posee el mutex, el hilo debe esperar hasta que esté disponible.

Después de realizar la modificación, el hilo libera el mutex mediante `pthread_mutex_unlock()`.

De esta manera, solo un hilo puede ejecutar la sección protegida en un determinado momento.

El resultado obtenido con mutex fue estable:

```
joaquin@LAPTOP-8G6CD4RU
lo 1$ ./race_mutex.exe
global_counter = 20
joaquin@LAPTOP-8G6CD4RU
lo 1$ ./race_mutex.exe
global_counter = 20
joaquin@LAPTOP-8G6CD4RU
lo 1$ ./race_mutex.exe
global_counter = 20
joaquin@LAPTOP-8G6CD4RU
lo 1$ ./race_mutex.exe
global_counter = 20
joaquin@LAPTOP-8G6CD4RU
lo 1$ ./race_mutex.exe
global_counter = 20
```

Condición de carrera

Una **condición de carrera** ocurre cuando dos o más hilos acceden simultáneamente a un recurso compartido y el resultado de la ejecución depende del orden en que los hilos realizan sus operaciones.

¿Por qué puede producir resultados diferentes al ejecutar varias veces el programa?

Porque los hilos se ejecutan concurrentemente y el sistema operativo puede cambiar entre ellos en diferentes momentos. Por lo tanto, el orden de las operaciones sobre `global_counter` puede variar de una ejecución a otra. La

condición de carrera puede provocar que algunas operaciones se sobrescriban entre sí. Por eso, el programa puede terminar mostrando un valor diferente de 20 y ese valor puede cambiar entre ejecuciones.

Sección crítica

Una **sección crítica** es una parte del programa que accede o modifica un recurso compartido y que debe ejecutarse de manera controlada para evitar que varios hilos interfieran entre sí.

En este caso, la sección crítica es la operación sobre:

global_counter++;

y

global_counter--;

porque ambos hilos modifican la misma variable compartida.

Generación del código ensamblador

Para analizar qué sucede a nivel de instrucciones de máquina se utilizó el comando:

```
gcc -S race_test.c
```

Este comando genera un archivo con extensión .s, que contiene el código ensamblador producido por GCC.

En este caso se analizaron dos archivos:

race_test.s

race_mutex.s

El primero corresponde a la versión sin mutex y el segundo a la versión con mutex.

Comparación del ensamblador con y sin mutex

En la versión sin mutex, el incremento aparece como:

```
.L3:
    movl    global_counter(%rip), %eax
    addl    $1, %eax
    movl    %eax, global_counter(%rip)
    addq    $1, -8(%rbp)
```

Mientras que el decremento aparece como:

```
.L7:
    movl    global_counter(%rip), %eax
    subl    $1, %eax
    movl    %eax, global_counter(%rip)
    addq    $1, -8(%rbp)
```

No existe ningún mecanismo que impida que el otro hilo acceda a `global_counter` mientras esta secuencia se está ejecutando.

En cambio, en la versión con mutex aparece:

```
.L3:
    leaq    mutex(%rip), %rax
    movq    %rax, %rdi
    call    pthread_mutex_lock@PLT
    movl    global_counter(%rip), %eax
    addl    $1, %eax
    movl    %eax, global_counter(%rip)
    leaq    mutex(%rip), %rax
    movq    %rax, %rdi
    call    pthread_mutex_unlock@PLT
    addq    $1, -8(%rbp)
```

```

.L7:
    leaq    mutex(%rip), %rax
    movq    %rax, %rdi
    call    pthread_mutex_lock@PLT
    movl    global_counter(%rip), %eax
    subl    $1, %eax
    movl    %eax, global_counter(%rip)
    leaq    mutex(%rip), %rax
    movq    %rax, %rdi
    call    pthread_mutex_unlock@PLT
    addq    $1, -8(%rbp)

```

Esto muestra a nivel de ensamblador cómo el mutex protege la sección crítica por medio de unlock y lock

Funciones movl, addl y subl

movl: se utiliza para mover/copiar un valor de 32 bits. En este caso, primero copia el valor de `global_counter` al registro `%eax`

addl: realiza una suma de 32 bits. En el incremento suma 1 al valor de `global_counter` que fue cargado previamente en `%eax`

subl: realiza una resta de 32 bits. En el decremento resta 1 al valor de `global_counter` que esta en `%eax`

En conclusion

En conclusión, los resultados obtenidos permitieron comprobar cómo una condición de carrera puede afectar el valor de una variable compartida cuando varios hilos acceden a ella simultáneamente. Al analizar el código ensamblador se observó que `movl`, `addl` y `subl` realizan la lectura, modificación y escritura de `global_counter`. La versión con mutex agrega `pthread_mutex_lock` y `pthread_mutex_unlock`, protegiendo estas operaciones como una sección crítica y evitando que ambos hilos modifiquen la variable al mismo tiempo. Por lo tanto, el uso del mutex permite controlar el acceso concurrente y obtener un comportamiento consistente del programa.